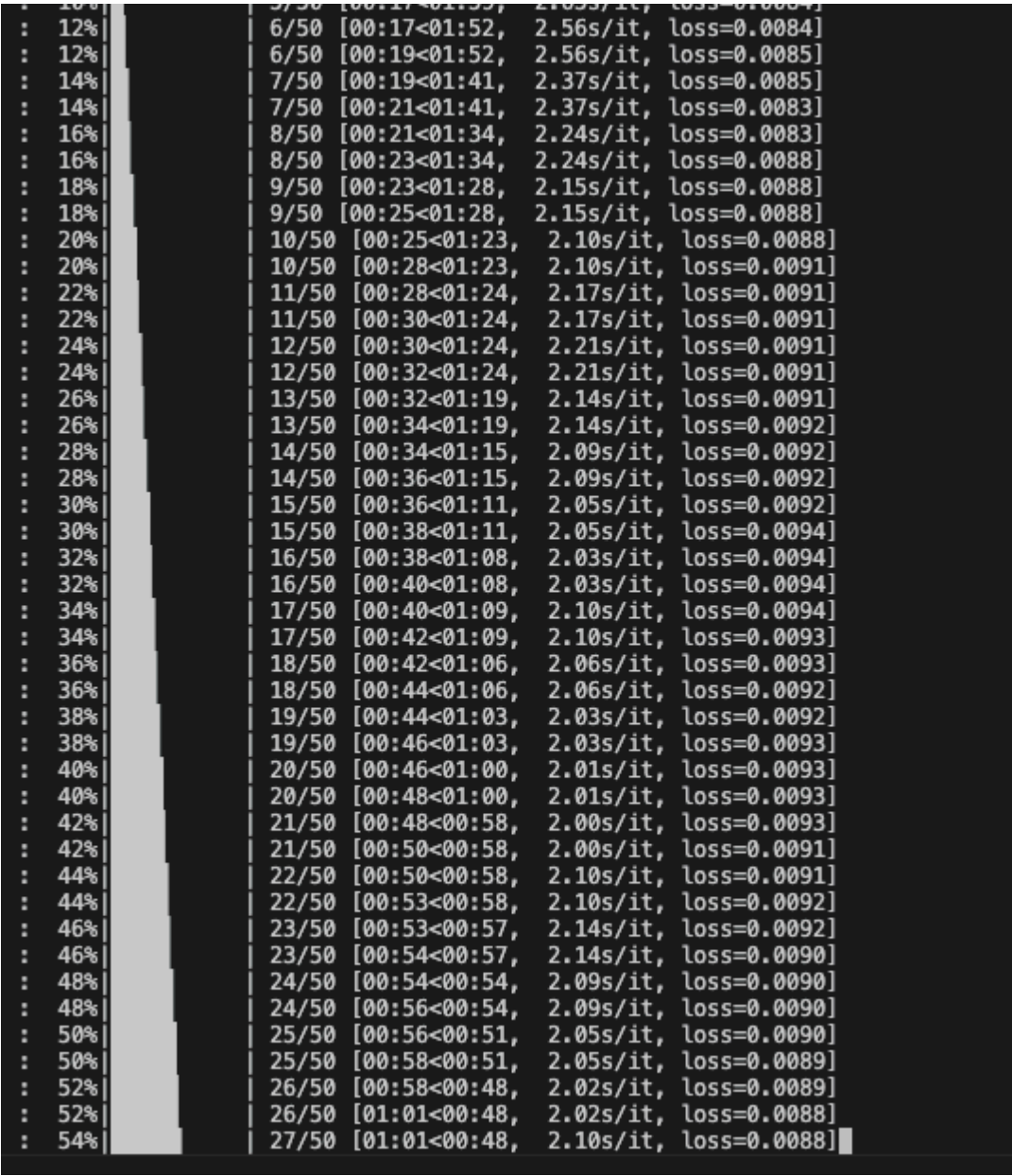


openpi

客户基于deepspeed-z2训练openpi 一开始在单卡上跑的 显存占用98% 很难添加优化手段 因此 先单机8卡跑一作为baseline

1.1单机8卡baseline



1.2 开启融合

```

: 70%|          | 35/50 [06:56<00:28, 1.87s/it, loss=0.0083]
: 70%|          | 35/50 [06:58<00:28, 1.87s/it, loss=0.0083]
: 72%|          | 36/50 [06:58<00:26, 1.87s/it, loss=0.0083]
: 72%|          | 36/50 [07:00<00:26, 1.87s/it, loss=0.0082]
: 74%|          | 37/50 [07:00<00:24, 1.87s/it, loss=0.0082]
: 74%|          | 37/50 [07:02<00:24, 1.87s/it, loss=0.0081]
: 76%|          | 38/50 [07:02<00:22, 1.87s/it, loss=0.0081]
: 76%|          | 38/50 [07:03<00:22, 1.87s/it, loss=0.0081]
: 78%|          | 39/50 [07:03<00:20, 1.87s/it, loss=0.0081]
: 78%|          | 39/50 [07:05<00:20, 1.87s/it, loss=0.0080]
: 80%|          | 40/50 [07:05<00:18, 1.87s/it, loss=0.0080]
: 80%|          | 40/50 [07:07<00:18, 1.87s/it, loss=0.0079]
: 82%|          | 41/50 [07:07<00:16, 1.87s/it, loss=0.0079]
: 82%|          | 41/50 [07:12<00:16, 1.87s/it, loss=0.0078]

```

1.3 关闭grad—ckpt

```

: 38%|          | 19/50 [05:56<00:49, 1.60s/it, loss=0.0091]
: 38%|          | 19/50 [05:57<00:49, 1.60s/it, loss=0.0091]
: 40%|          | 20/50 [05:57<00:46, 1.54s/it, loss=0.0091]
: 40%|          | 20/50 [05:59<00:46, 1.54s/it, loss=0.0089]
: 42%|          | 21/50 [05:59<00:42, 1.48s/it, loss=0.0089]
: 42%|          | 21/50 [06:00<00:42, 1.48s/it, loss=0.0089]
: 44%|          | 22/50 [06:00<00:40, 1.45s/it, loss=0.0089]
: 44%|          | 22/50 [06:02<00:40, 1.45s/it, loss=0.0088]
: 46%|          | 23/50 [06:02<00:38, 1.43s/it, loss=0.0088]
: 46%|          | 23/50 [06:03<00:38, 1.43s/it, loss=0.0088]
: 48%|          | 24/50 [06:03<00:36, 1.41s/it, loss=0.0088]
: 48%|          | 24/50 [06:04<00:36, 1.41s/it, loss=0.0088]
: 50%|          | 25/50 [06:04<00:34, 1.40s/it, loss=0.0088]
: 50%|          | 25/50 [06:06<00:34, 1.40s/it, loss=0.0087]
: 52%|          | 26/50 [06:06<00:33, 1.39s/it, loss=0.0087]
: 52%|          | 26/50 [06:07<00:33, 1.39s/it, loss=0.0088]
: 54%|          | 27/50 [06:07<00:31, 1.38s/it, loss=0.0088]
: 54%|          | 27/50 [06:08<00:31, 1.38s/it, loss=0.0087]
: 56%|          | 28/50 [06:08<00:30, 1.38s/it, loss=0.0087]
: 56%|          | 28/50 [06:10<00:30, 1.38s/it, loss=0.0087]
: 58%|          | 29/50 [06:10<00:28, 1.38s/it, loss=0.0087]
: 58%|          | 29/50 [06:11<00:28, 1.38s/it, loss=0.0086]
: 60%|          | 30/50 [06:11<00:27, 1.37s/it, loss=0.0086]
: 60%|          | 30/50 [06:13<00:27, 1.37s/it, loss=0.0085]
: 62%|          | 31/50 [06:13<00:26, 1.37s/it, loss=0.0085]
: 62%|          | 31/50 [06:14<00:26, 1.37s/it, loss=0.0084]
: 64%|          | 32/50 [06:14<00:24, 1.38s/it, loss=0.0084]
: 64%|          | 32/50 [06:15<00:24, 1.38s/it, loss=0.0083]
: 66%|          | 33/50 [06:15<00:23, 1.37s/it, loss=0.0083]

```

loss收敛正常 训练时显存88%-89%

如果开启triton_fusion 会loss Nan

单机8卡	baseline	+torch.compile max-autotune-no-cudagraphs	+关grad_ckpt
稳态迭代耗时	2.01	1.87	1.36

当前脚本scripts/tc_cloud/train_tc.sh 已经合入最优配置 保留原始脚本
scripts/tc_cloud/train_tc.sh-bak

选项：开启offload：config/zero2_offload.json（当前显存占用90% 未启用

单卡关grad_ckpt会oom offload也没用

客户要求必须单卡训练 修改优化思路

2. 合入

2.1 原生实现

2.1.1 基础算法实现复习

这里主要是从底层逻辑分析注意力的实现 传统实现和FA的实现在 [Qwen3.5-0.8B训练](#) 的1.3.1部分已经分析的较为详细 这里简单回顾 FA对于注意力传统实现的加速是基于分块和融合实现的 降低的不是计算量 而是显存读写量 这样计算快很多 与此同时 FA还有以下三个特点：

- **支持原生GQA**：直接吃 8:1 的 Q/KV 比例 不复制 KV
- **支持 causal 和 bidirectional**：也就是单双向都支持
- **对 mask 有要求**：FA2 的 base 版本（不是 varlen 版本）基本不能吃复杂 mask 主要靠 padding mask + varlen 打包来处理有效/无效位

简单说一下这三个点

第一个点——GQA是分组查询注意力 也就是

- **Hq (num_attention_heads) = 8**：查询头数 8 个
- **Hkv (num_key_value_heads) = 1**：键值头数只有 1 个

8 个查询头共用 1 组 KV 好处是大幅减少 KV 的体积（显存和计算量）对应的传统做法是 MHA（Q头数=KV头数）

有两种实现方法 一个是把KV复制8份 假装成传统的MHA再算 简单但是费时间 还有一种就是 FlashAttention 等融合算子支持直接用 8:1 的比例计算 不复制 `enable_gqa` 就是干这个的

第二个点——支持单双向

顾名思义就是单个的token是能只能看一个方向的还是两个方向的其他token都能看到 Openpi模型的前缀后缀都是双向的 所以 `is_causal=False`

第三个点——对Mask有要求

虽然当前是每个token都能看到其他所有token 但是实际上序列里有填充（padding）token——为了让一个 batch 里长度不一的样本能拼成矩形 短的样本末尾补占位符 这些占位符不携带信息 不应该参与注意力的计算 Mask就是用来标记哪些位置有效、哪些是 padding的张量 有两种主流表示法

- Additive mask: 形状 `[B, H, Sq, Sk]` 或 `[B, Sq, Sk]` 有效位置 = 0 无效位置 = 一个很大的负数, 比如 `-1e30` 数学上 `scale(QK^T) + mask` 无效位置加上 `-1e30` softmax 后就变成 ~ 0 等于"不参与"
- bool mask: 形状 `[B, Sq]` 或 `[B, Sk]` dtype是0和1 0表示无效 1 表示有效 它只表达这个整行/整列有没有效

以上是FA2的特点和介绍 提到了FA2（base版）不擅长吃Mask 说具体点就是只会对因果掩码（causal mask就是单向掩码）做了专门的优化 内核可以通过一个简单的 `is_causal=True` 参数来高效处理 不需要外部传入复杂的掩码张量 除了这个内置支持的因果掩码外 FA2 对于其他任何“非标准”的掩码形式 都没有在 kernel 内部做实现 而这里Openpi的 `is_causal=False` 那么怎么更好的处理这里的padding呢 答案是 varlen（variable length变长）模式

不去处理带 padding 的方阵 而是**先把有效 token 从方阵里抽出来、打包成一维长条** 记录每个样本的有效长度（用 `cu_seqlens` 累计长度数组表示） 然后对这条紧凑的一维序列一次性跑 Flash 输出再散回原方阵形状 这一点在 [Dreamzero部署及性能优化](#) 的2.7部分提到过 是该模型优化的一个重点 可以简单表示为

代码块

```
1  原始: [有效 有效 有效 PAD PAD]      ← 方阵里有空位
2  压缩: [有效 有效 有效]                ← 紧凑一维 + 长度记录 [0,3]
3  送 Flash → 输出 [有效 有效 有效]
4  散回: [有效 有效 有效 PAD PAD]
```

对应的函数名flash_attn_varlen_func

补充一下SDPA 这个对于基于哪一种注意力实现尤为关键 是 PyTorch 提供的一个"统一入口"注意力算子 特点是聪明的后端选择——不用管底层用什么实现 PyTorch 自动从下面三个后端里挑:

1. **Flash Attention 后端**: 最快 但不支持非空 mask（也要求特定 dtype/shape）
2. **Memory-Efficient 后端**: 次选 支持 mask 速度也还可以 不过我没用过
3. **Math 后端**: 最朴素的实现 一个个算吧就

SDPA 的 fallback 链是后面所有问题的源头：

比如给它一个非空 `attn_mask` → Flash 后端拒绝 → 看有没有 mem-efficient → 没有 → 退到 **Math** 后端 → 慢。

2.1.2 Openpi中注意力的原生实现——eager

HuggingFace Transformers 框架中每个模型都有一个配置项 `_attn_implementation`

- `"eager"`：手写注意力（最朴素，永远可走，最慢的兜底）
- `"sdpa"`：走 PyTorch SDPA 入口
- `"flash_attention_2"`：走 HF 封装的 FA2 入口
- `"flex_attention"`：灵活掩码后端（较新）
- `"flash_attention_3"`：FA3（更新版）

在 OpenPI/pi05 里，这个值来自一个上层配置 `attention_implementation`，取值是 `"eager"`、`"fa2"`、`"flex"`

pi05/paligemma_with_expert_pi05.py:157

代码块

```
1 if self.attention_implementation not in ["eager", "fa2", "flex"]:  
2     raise ValueError(...)
```

而这里用的到底是哪个呢 看 HF 这版 transformers (4.53.2) 里 Gemma 的 Attention 实现 原码是这么写的

代码块

```
1 class GemmaAttention(nn.Module):  
2     def forward(self, hidden_states, position_embeddings, attention_mask, ...):  
3         # 线性投影得到 Q/K/V  
4         query_states = self.q_proj(hidden_states).view(...).transpose(1, 2)  
5         key_states = self.k_proj(hidden_states).view(...).transpose(1, 2)  
6         value_states = self.v_proj(hidden_states).view(...).transpose(1, 2)  
7
```

```

8         # RoPE 旋转位置编码
9         query_states, key_states = apply_rotary_pos_emb(query_states,
10                                                         key_states, cos, sin)
11
12         # 处理 KV cache (prefill 时拼接过去的 KV)
13         if past_key_value is not None: ...
14
15         attention_interface = eager_attention_forward          # 默认 eager
16         if self.config._attn_implementation != "eager":
17             attention_interface =
18             ALL_ATTENTION_FUNCTIONS[self.config._attn_implementation]
19
20         attn_output, attn_weights = attention_interface(self, query_states,
21                                                         key_states, value_states,
22                                                         attention_mask,
23                                                         dropout=0.0, scaling=..., **kwargs)
24         # 投影回原维度
25         attn_output = self.o_proj(attn_output.reshape(...))

```

这里第16行表明 只要配置写 `"eager"` 就走 `eager_attention_forward`

而非 eager 时查一个全局字典 `ALL_ATTENTION_FUNCTIONS` 按名字走对应实现 这个字典注册了 `sdpa fa2`等键

pi05/paligemma_with_expert_pi05.pybak:57这里的原本实现是

代码块

```
1 attention_implementation: str = "eager",
```

这里走的eager 根本不进fsdp字典查询

2.2 从eager到sdpa的FA

2.2.1 接口设置

启动脚本传参直接指定注意力后端 `+pi0_config.attention_implementation=fa2`

在paligemma_with_expert_pi05.py中添加

代码块

```

1 #####dk_test#####
2 if config.attention_implementation == "fa2":
3     for model in (self.paligemma.language_model, self.gemma_expert.model):
4         model.config._attn_implementation = "flash_attention_2" # noqa: SLF001
5         for layer in model.layers:
6             layer.self_attn.is_causal = False
7 #####

```

然后Hydra 把值注入到运行期 config 对象 在modeling_pi05.py构造 PaliGemma 时把上层 config 的值fa2传下去

代码块

```

1 paligemma_with_export_config = PaliGemmaWithExpertConfig(
2     attention_implementation=self.config.attention_implementation, # 传入的是 "fa2"
3     ...
4 )

```

虽然attention_implementation: str = "eager"还在 但是只要传了别的后端这个就被覆盖了

当然 这只是接口的修改 刚才说了 如果就算之前走sdpa了也轮不到fa 因为输入的注意力掩码的问题 这就到了第二个改动

2.2.2 4D additive mask → 2D padding mask

代码块

```

1 #####dk_test#####
2 split_forward = (inputs_embeds[0] is None) != (inputs_embeds[1] is None)
3 if (
4     self.config.attention_implementation == "fa2"
5     and split_forward
6     and attention_mask is not None
7     and attention_mask.ndim == 4
8 ):
9     attention_mask = attention_mask[:, 0].eq(0).any(dim=-2)
10 #####

```

if代码块描述的是输入的情况 也就是原本的输入 4D加性注意力mask 这个fa处理不了

代码块


```
1 attention_mask = attention_mask[:, 0].eq(0).any(dim=-2)
```

这句是精髓

- `attention_mask[:, 0]`：从 `[B, 1, Sq, Sk]` 抽出 `[B, Sq, Sk]`（那个 1 是 head 维 所有头共用同一掩码所以直接降掉 这是个冗余的维度）
- `.eq(0)`：数据类型把加性mask翻译成 bool `0` 在加性语义里是有效 所以 `.eq(0)` 得到 True 表示有效
- `.any(dim=-2)`：在 Sq 维上做“存在任一有效位”的折叠 得到 `[B, Sk]`

完成了从维度到掩码数据类型的转换 也就是2.1.1说的 转换成了fa能够处理的掩码形式 为什么可以压缩为2D的padding mask?

因为在 split_forward 模式下（一次只跑 prefix 或 suffix）块内都是双向注意力：prefix 块内谁都能看谁 suffix 块内也是 掩码真正要表达的只是“哪些列是 padding 不要看”这恰好能用 2D padding mask 表达

2.2.3 prefix KV cache 对齐 dtype

FA2 对 dtype 一致性要求比 Math 严格（要 BF16 全程 BF16 / fp32 全程 fp32）如果 KV cache 之前缓存的是别的 dtype（比如 fp32）FA2 内核会拒绝 所以这里在梯度检查点重算前 把cache里的K/V都转成当前输入dtype

代码块

```
1 if past_key_values is not None and not use_cache and
   model.config._attn_implementation == "flash_attention_2":
2     past_key_values.key_cache = [x.to(inputs_embeds.dtype) for x in
   past_key_values.key_cache]
3     past_key_values.value_cache = [x.to(inputs_embeds.dtype) for x in
   past_key_values.value_cache]
```

2.2.4 整理

改动内容上来说 接口设置让注意力后端能走fa 注意力掩码的修改让fa可以走的通 都是主动行为 最后一步对齐数据类型是保障不出问题

这样转换的收益在于

- 从拆 5 个算子 → 1 个融合算子：原本 $QK^T \rightarrow +mask \rightarrow scale \rightarrow softmax \rightarrow PV$ 每一次都意味着显存读写 fa的优势前面说过了
- GQA：从复制 KV → 原生 8:1：原生是复制kv凑够8个

代码块

```
1  if hasattr(module, "num_key_value_groups"):
2      key = repeat_kv(key, module.num_key_value_groups)
3      value = repeat_kv(value, module.num_key_value_groups)
```

而FA2 内核原生支持 GQA 不复制的话省显存又省计算

收益体现：

场景	Math SDPA	FA2
prefix checkpoint attention	20.962 ms	7.330 ms
suffix checkpoint attention	4.098 ms	2.101 ms

2.2.5 铺垫：转换为二维布尔注意力掩码之后发生了什么？

transformers/integrations/flash_attention.py中

代码块

```
1  if attention_mask is not None:
2      query_states, key_states, value_states, indices_q, cu_seq_lens,
        max_seq_lens = _upad_input(
3          query_states, key_states, value_states, attention_mask, query_length,
        _unpad_input
4      )
5      attn_output_unpad = _flash_attn_varlen_func(
6          query_states, key_states, value_states,
7          cu_seq_lens_q=cu_seq_lens_q, cu_seq_lens_k=cu_seq_lens_k,
8          max_seq_len_q=..., max_seq_len_k=...,
9          softmax_scale=softmax_scale, causal=causal, **flash_kwargs,
10     )
11     attn_output = _pad_input(attn_output_unpad, indices_q, batch_size,
        query_length)
```

- _upad_input：拿着 [B, Sk] 这个2D的padding mask 把方阵里有效 token 抽出来打包成一维紧凑序列 输出 cu_seq_lens（累计长度）和 max_seq_lens（最大长度）
- _flash_attn_varlen_func：对这条紧凑序列跑一次 varlen Flash 内核

3. _pad_input: 把结果散回 [B, Sq, H, D] 方阵

这就是2.1.1中说到的flash_attn_varlen_func的实际行为

也就是说这一条龙的处理:

先设置fa2的接口 再转换注意力掩码 再对齐数据类型 再保留有效部分打包为变长序列进行attention计算 计算完成后对输出结果进行散回

2.3 玺君的优化分析



 fused_sdpa.py

优化逻辑是

代码块

```
1 scale = ...
2 attention_mask = _trim_mask(attention_mask, key.shape[-2])
3 if is_causal is None:
4     is_causal = query.shape[2] > 1 and attention_mask is None and
5         getattr(module, "is_causal", True)
6 if attention_mask is None and is_causal:
7     attn_output = _sdpa_flash(query, key, value, scale, causal=True)
```

```

8  else:
9      attn_output = flash_sdpa_with_mask(query, key, value, attention_mask,
      scale)
10
11  attn_output = attn_output.transpose(1, 2).contiguous()
12  return attn_output, None

```

先判断：无 mask 且 causal 就直接 Flash 否则进 `flash_sdpa_with_mask` 处理 mask

后者代码块如下

代码块

```

1  def _analyze_mask(attention_mask, sq, sk) -> _Plan:
2      keep = _keep_bool_from_mask(attention_mask, sq, sk)  # 加性 → bool
3      keep_q = keep.any(dim=-1)  # 每个 query 是否有任一有效 key
4      keep_k = keep.any(dim=-2)  # 每个被 key 是否被任一 query 有效
5      expected = keep_q[:, :, None] & keep_k[:, None, :]  # "理想化"的 q-k 外积结
      构
6
7      # 不符合"key/query padding 外积"的结构 → math 分支
8      if not torch.equal(keep, expected):
9          return _Plan(kind="math", ...)
10
11     q_len = keep_q.sum(dim=1)
12     k_len = keep_k.sum(dim=1)
13
14     # 全员有效 → dense 分支
15     if (q_len == sq).all() and (k_len == sk).all():
16         return _Plan(kind="dense", ...)
17
18     # 有 padding → varlen 分支
19     if flash_attn_varlen_func is None:
20         return _Plan(kind="math", ...)  # 没有 varlen 库就退 math
21
22     # 打包下标和累计长度
23     q_b, q_s = keep_q.nonzero(as_tuple=True)
24     k_b, k_s = keep_k.nonzero(as_tuple=True)
25     cu_q = F.pad(q_len.cumsum(0), (1, 0)).to(torch.int32)
26     cu_k = F.pad(k_len.cumsum(0), (1, 0)).to(torch.int32)
27     return _Plan(kind="varlen", q_b=q_b, q_s=q_s, k_b=k_b, k_s=k_s,
28                     cu_q=cu_q, cu_k=cu_k, max_q=..., max_k=..., sq=sq, sk=sk)

```

三类 plan:

- **dense** : mask 全是有效（也就是没有 padding）直接 `flash_attn_func` 一次
- **varlen** : mask 是 key/query padding 外积结构（这是 OpenPI 想要的）用 `flash_attn_varlen_func` 打包有效 token 跑
- **math** : mask 是真正的"部分可见"结构（比如真 prefix-LM 的 prefix 看不到 suffix）varlen 救不了回退到 `F.scaled_dot_product_attention` 的 math 后端

然后如果是varlen 走的是

代码块

```
1 def _flash_varlen(q, k, v, plan, scale):
2     q_pack = q[plan.q_b, :, plan.q_s, :].contiguous()
3     k_pack = k[plan.k_b, :, plan.k_s, :].contiguous()
4     v_pack = v[plan.k_b, :, plan.k_s, :].contiguous()
5     out_pack = flash_attn_varlen_func(q_pack, k_pack, v_pack,
6                                     plan.cu_q, plan.cu_k, plan.max_q,
7                                     plan.max_k,
8                                     dropout_p=0.0, softmax_scale=scale,
9                                     causal=False)
10    out = q.new_zeros(q.shape)
11    out[plan.q_b, :, plan.q_s, :] = out_pack
12    return out
```

这和2.2.5做的事情是一样的

2.4 为什么2.2和2.3的优化不能叠加

第一 优化的对象是sdpa和eager 但是2.2已经指定了注意力后端走fa2 压根没接上

- HF 的分派逻辑是查 `ALL_ATTENTION_FUNCTIONS` 这个字典 key 是 `"flash_attention_2"` 同事的 `patch_hf_gemma_sdpa` 替换的是 `"sdpa"` 和 `"eager"` 这两个 key 没有替换 `"flash_attention_2"`

第二 就算对上也没用 因为2.3做的就是让注意力后端走fa2 只不过是手写的函数 没有用transformer包里的varlen代码实现 本质上一样

- Transformer 包里的 HF 原生 FA2 forward（2.2的方法）已经做了 `_upad_input`（打包）+ `flash_attn_varlen_func`（计算）+ `_pad_input`（散回）这一套组合拳 2.3是自己写的 没区别

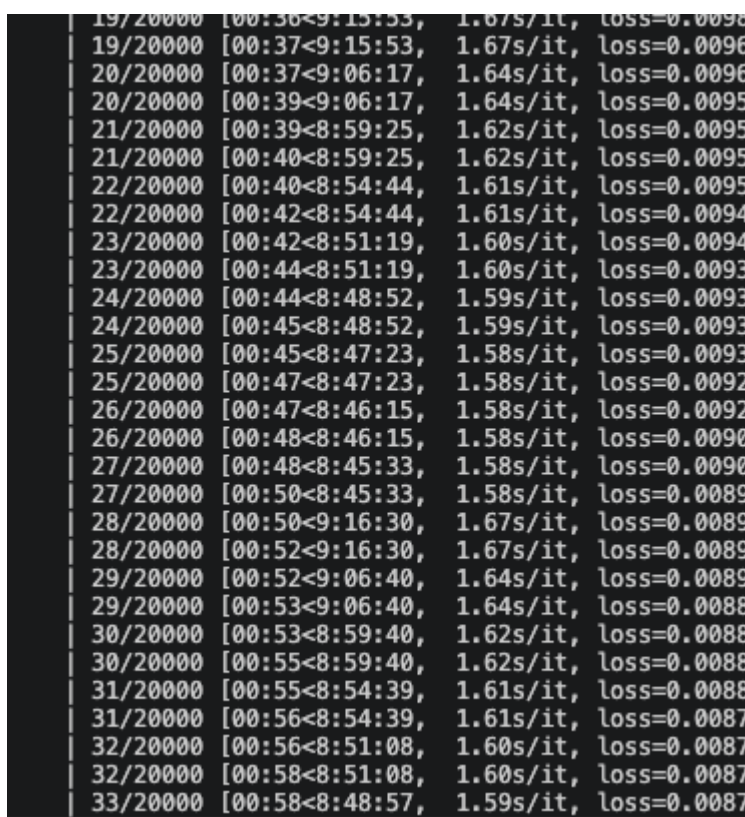
综上所述 两个优化没法叠加

2.5 算子融合

export HA_USE_RMSNORM=1

export HA_USE_ROPE=1

export HA_USE_FUSED_GELU_MUL=1 后性能进一步提升



19/20000	[00:36<9:15:53,	1.67s/it,	loss=0.0096
19/20000	[00:37<9:15:53,	1.67s/it,	loss=0.0096
20/20000	[00:37<9:06:17,	1.64s/it,	loss=0.0096
20/20000	[00:39<9:06:17,	1.64s/it,	loss=0.0095
21/20000	[00:39<8:59:25,	1.62s/it,	loss=0.0095
21/20000	[00:40<8:59:25,	1.62s/it,	loss=0.0095
22/20000	[00:40<8:54:44,	1.61s/it,	loss=0.0095
22/20000	[00:42<8:54:44,	1.61s/it,	loss=0.0094
23/20000	[00:42<8:51:19,	1.60s/it,	loss=0.0094
23/20000	[00:44<8:51:19,	1.60s/it,	loss=0.0093
24/20000	[00:44<8:48:52,	1.59s/it,	loss=0.0093
24/20000	[00:45<8:48:52,	1.59s/it,	loss=0.0093
25/20000	[00:45<8:47:23,	1.58s/it,	loss=0.0093
25/20000	[00:47<8:47:23,	1.58s/it,	loss=0.0092
26/20000	[00:47<8:46:15,	1.58s/it,	loss=0.0092
26/20000	[00:48<8:46:15,	1.58s/it,	loss=0.0090
27/20000	[00:48<8:45:33,	1.58s/it,	loss=0.0090
27/20000	[00:50<8:45:33,	1.58s/it,	loss=0.0089
28/20000	[00:50<9:16:30,	1.67s/it,	loss=0.0089
28/20000	[00:52<9:16:30,	1.67s/it,	loss=0.0089
29/20000	[00:52<9:06:40,	1.64s/it,	loss=0.0089
29/20000	[00:53<9:06:40,	1.64s/it,	loss=0.0088
30/20000	[00:53<8:59:40,	1.62s/it,	loss=0.0088
30/20000	[00:55<8:59:40,	1.62s/it,	loss=0.0088
31/20000	[00:55<8:54:39,	1.61s/it,	loss=0.0088
31/20000	[00:56<8:54:39,	1.61s/it,	loss=0.0087
32/20000	[00:56<8:51:08,	1.60s/it,	loss=0.0087
32/20000	[00:58<8:51:08,	1.60s/it,	loss=0.0087
33/20000	[00:58<8:48:57,	1.59s/it,	loss=0.0087

2.5.1 RMSNorm 融合（HA_USE_RMSNORM=1）

Gemma 每层有 RMSNorm（比 LayerNorm 简单一点）： $x / \sqrt{\text{mean}(x^2) + \epsilon} * \gamma$ 。原版是几个 PyTorch 算子拼出来的：square $\rightarrow$ mean $\rightarrow$ add $\epsilon \rightarrow$ rsqrt $\rightarrow$ mul $\rightarrow$ mul γ

每一步都要读写 HBM hyperacc 把这一串融合成一个 kernel：读 x 一次、写 y 一次，中间全在 SRAM 算完。每层前后各一次 norm $\times$ 26 层 $\times$ 两个 Gemma = 大量省 HBM 读写。

2.5.2 RoPE 融合（HA_USE_ROPE=1）

RoPE（旋转位置编码）把 Q 、 K 做旋转： $\text{rotate_half}(x) * \cos + \text{flip}(x) * \sin$ 。原版是构造 $\cos/\sin \rightarrow$ 拆 $\text{rotate_half} \rightarrow$ 两次 mul + cat。每层每步一次。

hyperacc 的 rotary_embedding_hip kernel 把这串融合成单 kernel， Q/K 各一次读、一次写。Gemma 26 层 $\times$ 两个 Gemma $\times Q$ 和 K 各一次 = 几百次 RoPE/step，这串融合掉省的量也不小。

2.5.3 Fused GELU $\times$ Mul（HA_USE_FUSED_GELU_MUL=1）

Gemma 的 MLP 结构是：

gate = gate_proj(x)

up = up_proj(x)

y = gelu_tanh(gate) * up ← 这一行

原版两个算子：先 gelu_tanh(gate)（一个 elementwise + tanh 近似），再 * up（逐元素乘），中间张量写回 HBM 再读出来。

hyperacc 的 fused_gelu_mul 把 gelu(gate) * up 融合到一个 kernel：读 gate 和 up 一次、写出 y 一次，gelu 和 mul 在寄存器里算完。这个张量是 $B \times S_q \times \text{intermediate_size}$ （intermediate_size 比较大），中间激活读写是大头，融合掉的收益很明显。

	BW1000	BW1000优化后	A800
单次迭代耗时	2.07s/it	1.53s/it	1.57s/it